Содержание

Power Systems Agent Benchmark: Bringing Executable Evaluation to AI Agents on Electric Power Engineering Tasks **1**
Abstract . . . 1
1. Introduction . . . 2
2. Related Work . . . 3
3. Requirements for a Power-Systems Agent Benchmark . . . 4
4. Architecture . . . 4
4.1 Artifact contract . . . 4
4.2 Agent-agnostic launch and harness robustness . . . 4
4.3 Evaluator maturity ladder . . . 5
4.4 The hidden split as a generator, not as files . . . 5
5. Task Set . . . 5
5.1 Domain vetting of the set . . . 6
6. Evaluators and Metrics . . . 6
7. Reproducibility . . . 6
8. Experiments . . . 6
8.1 Canonical round (public vs. hidden) . . . 7
8.2 Failure taxonomy: capability versus specification . . . 7
8.3 Evaluation integrity: an agent that reached the oracle . . . 8
9. Limitations . . . 8
10. Conclusion . . . 8
Appendix A. Task Catalog . . . 9
Appendix B. Experiment Artifacts . . . 9

Power Systems Agent Benchmark: Bringing Executable Evaluation to AI Agents on Electric Power Engineering Tasks

Preprint draft v2 (2026-06). English version for arXiv (eess.SY; cs.AI). Word-for-word Russian companion: `paper_ru_v2.md`*. Annotated bibliography:* `bibliography.md`*; BibTeX:* `references.bib`*.*

Abstract

Executable evaluation — having an agent act and then checking the consequences of that action with a program rather than grading its prose — is by now a well-established practice for tool-using AI agents, from software repair (SWE-bench) to tool-and-API interaction (τ-bench) and terminal work (terminal-bench). Electric power engineering has so far stood largely outside this practice: agents in the domain are still evaluated by text question answering or by LLM-as-a-judge, and the few domain benchmarks either score free text (ElecBench) or cover a single problem type (PFBench, power flow only). We bring the executable paradigm to broad power-engineering agent evaluation. In the Power Systems Agent Benchmark an agent receives a structured task, returns a structured solution, and a deterministic evaluator recomputes the engineering quantities, checks the physical and operational constraints, and reports a feasibility flag, a normalized score, and an explicit list of violations. The public split spans 41 task families — power-flow and network analysis, short-circuit and protection, stability and grid-code compliance, DER/EV and storage, microgrid dispatch, reliability and restoration, power quality and standards, OT cybersecurity, and forecasting — each grounded in a cited source and a standard. Rather than ship a static hidden split, which ages and leaks, we publish a deterministic generator that derives hidden cases from a private seed with automatic solvability checks, so the mechanism is auditable while the numbers stay private. Our central methodological result is that running a small matrix of agents and reading an identical failure of all of them as

a defect signal is a practical quality-control procedure for executable benchmarks: it surfaced ten specification defects and, most tellingly, a latent physics bug inside one of our own evaluators that the benchmark's self-consistency check could not see. On clean specifications the public and hidden splits agree on the ranking and yield comparable scores, so we position hidden evaluation as prospective contamination resistance rather than a one-round difficulty gap. The evaluators are compact deterministic surrogates today; replacing their internals with simulator-backed implementations (pandapower, OpenDSS) is a roadmap that leaves the task contract unchanged.

1. Introduction

The way we evaluate AI agents has been quietly changing. As agents have moved from answering questions to acting — reading data, writing helper code, calling tools, and committing to decisions — the most informative benchmarks have stopped grading what an agent says and started checking what its actions do. SWE-bench runs the project's test suite against the agent's patch; τ-bench compares the final state of a database against an annotated goal; terminal-bench inspects the effect of commands on a real shell. This executable style of evaluation is now established practice, and for good reason: it measures outcomes rather than persuasiveness, and it resists the biases that attach to a language model asked to judge another model's prose.

Electric power engineering has largely missed this shift. Agents are arriving in the domain — for fault analysis, dispatch, interconnection studies — but the benchmarks used to assess them still lean on reference-answer question answering or on LLM-as-a-judge grading, and the few domain-specific efforts narrow the scope to keep scoring tractable: ElecBench evaluates dispatch reasoning largely through text and judges, and PFBench applies programmatic checking but only to power flow. The mismatch matters here more than elsewhere, because a power-system answer can be fluent and still be wrong in ways that text cannot reveal. A proposed relay setting can read as authoritative and fail to be selective; a dispatch can be well argued and violate a generator's limits; a restoration plan can be coherent and de-energize the wrong feeder. The error surfaces not as a phrasing but as an overloaded line, an inadmissible voltage, or a mislocated fault — and only a check that recomputes the physics can catch it.

This paper brings executable evaluation to power-engineering agents. We do not claim the executable idea, nor the notion of a tool-using agent; both are inherited from the benchmarks above. What we contribute is the adaptation of that idea to a domain that has lacked it, at breadth, together with the machinery that adaptation requires and a methodology for keeping it honest. Concretely, each task publishes structured inputs; the agent returns a machine-readable solution; and an independent evaluator recomputes the target quantities, checks the hard constraints, and returns a feasibility flag, a normalized score, and an explicit list of violations. The agent's free-text rationale is retained but deliberately kept out of the scoring loop — it is a diagnostic, not a basis for credit.

Five contributions follow from this design. First, we instantiate the executable contract for power engineering as a uniform task/submission/evaluation interface with JSON schemas for every artifact and an agent-agnostic launch protocol; the contribution is not the contract idea but its concrete, domain-spanning realization. Second, we assemble a balanced public set of 41 executable task families across eight domain groups, each carrying traceable provenance and a bilingual presentation, so that the benchmark's coverage and grounding — not a single showcase problem — are what a reader can inspect. Third, we replace the usual static hidden split with a deterministic per-family generator: hidden cases are derived from a private seed on the evaluation host, and a draw is kept only if the family baseline solves it and the evaluator confirms feasibility, which makes the leakage-prone hidden set auditable in mechanism but private in numbers. Fourth, and most importantly, we show that a small agent matrix doubles as a quality-control instrument: when every agent fails a task in the same way, the defect is usually in the specification, not the agents, and acting on this criterion let us find and fix ten specification defects. Fifth, the strongest instance of that criterion turned inward — it exposed a latent physics bug inside one of our own evaluators (an off-by-one in a fault-current-indicator model) that the benchmark's baseline-equals-evaluator self-consistency check was structurally unable to detect, because the baseline had inherited the same error; three independent agents applying the correct physics were what revealed it.

Underlying all of this is a piece of infrastructure that fair agent comparison turns out to require, and which we treat as a reusable contribution in its own right: a harness that isolates each run so an agent cannot reach the oracle, tolerates malformed submissions without aborting, survives evaluator crashes, and separates genuine agent failures from provider flakiness through a narrow retry policy that reports raw and retry-normalized metrics side by side (Section 4).

2. Related Work

Our work sits at the meeting point of three lines of research, and our debts to each are easiest to state by where we follow them and where we depart.

The first line is executable agent evaluation. SWE-bench [jimenez2024swebench] and the earlier Codex/HumanEval pass@k construction [chen2021codex] established that an agent's output can be judged by running it; τ-bench [yao2024taubench] generalized the idea to tool-and-user interaction by comparing end states and introduced pass^k as a reliability measure; terminal-bench [terminalbench2025] and MLE-bench [chan2024mlebench] extend it to shell work and machine-learning engineering, while general suites such as AgentBench [liu2024agentbench], GAIA [mialon2023gaia], and WebArena [zhou2024webarena] probe broad tool use. We adopt this paradigm wholesale; our only quarrel with it is one of coverage, since none of these targets power-system engineering and its physical constraints.

The second line motivates why we keep an LLM out of the scoring loop. Judging with a language model [zheng2023judging] is convenient but carries documented biases — position, length, and self-preference among them [wang2023fairevaluators, gu2024judgesurvey, li2024generationjudgment] — and these are most dangerous exactly when a wrong answer is fluent, which is the common case in engineering. We therefore score by deterministic recomputation and demote the model's rationale to a diagnostic.

The third line is domain-specific. Surveys [llmpowersurvey2025, agenticpowersurvey2025] document a rapid influx of LLM and agent methods into power systems with no settled way to evaluate them. The closest effort to ours is PowerAgentBench [poweragentbench], the benchmark component of the PowerAgent ecosystem at Harvard SEAS [zhang2025poweragent], which likewise pairs a public/hidden split with a hidden evaluator that recomputes physical validity, but over three task levels on the IEEE 39-bus system rather than a broad family set, and without a published paper at the time of writing. ElecBench [zhou2025elecbench] scores dispatch scenarios largely through text and judges, and PFBench [pfbench2025] applies programmatic checking but only to power flow. The production agents now appearing — GridMind [gridmind2025], X-GridAgent [xgridagent2025], Grid-Mind CIA [gridmindcia2026], PFAgent [pfagent2026] — are the intended subjects of a benchmark like ours. We differ from these neighbors in breadth (41 families across eight domain groups), in generating hidden cases from a seed rather than storing them, and in the explicit quality-control loop.

Two further bodies of work shape specific design choices. Reinforcement-learning grid environments, principally Grid2Op/L2RPN [marot2020l2rpn, marot2021retrospective], offer a mature setting for sequential topology control; we address the complementary problem of single-shot engineering decisions across many subtasks under an agent-agnostic contract, and we leave the simulators that such environments rely on — pandapower [thurner2018pandapower], MATPOWER [zimmerman2011matpower], OpenDSS [dugan2011opendss], PowerModels/PGLib [coffrin2018powermodels, babaeinejadsarookolaee2019pglib], ANDES [cui2021andes], PyPSA [brown2018pypsa] — as the substrate into which our checkers can later be swapped without changing the task format. Finally, the literature on benchmark integrity informs our hidden split: because training-data contamination inflates results [contaminationsurvey2025, statictodynamic2025], dynamic and refreshed benchmarks have become the standard response [kiela2021dynabench, white2024livebench, jain2024livecodebench], and our seed-based generator is a prevention mechanism of the same family — we publish how cases are made rather than the cases. The harness, in turn, follows the prescriptions of [kapoor2024agentsmatter] on cost, reproducibility, and the separation of failure classes, and respects the limits

of best-of-N established in [brown2024monkeys].

3. Requirements for a Power-Systems Agent Benchmark

Before describing the system it is worth setting down what a benchmark of this kind must satisfy, since several of our design decisions are direct consequences of these requirements. A solution must be verified by code rather than by string matching (executability), and that verification must recompute physical quantities and check constraints rather than compare against stored text (physical verifiability). Public cases must support development while hidden cases support fair comparison, with hidden answers excluded by construction rather than by policy (public/hidden separation). Launching an agent must not depend on a particular model, CLI, or framework — the contract reduces to "read files, write a file" (agent agnosticism). Every task must declare where it comes from, with a confidence level (provenance), and every result must be machine-readable — `feasible`, `score $\in$ [0,1]`, `violations`, and metrics (machine-readable result). The benchmark must be able to grow harder without breaking compatibility, so that moving from a formula surrogate to a simulator-backed check alters an evaluator's internals but not the task or submission format (difficulty growth without a contract change). To these seven, which we set out in advance, the experiments later add an eighth that we did not anticipate: the benchmark must include a procedure for telling specification defects apart from genuine capability differences (specification-quality control, Section 8.2).

4. Architecture

4.1 Artifact contract

The benchmark is organized around a single linear contract,

```
task.json → agent (any CLI) → submission.json → evaluator → evaluation-result.json
```

with a JSON schema fixing each artifact. A `task.json` carries the task identity and family, the public inputs, the submission schema, the hard constraints, the metrics, the provenance, and a bilingual presentation block; a `submission.json` carries the agent's structured answer and an optional rationale; and an `evaluation-result.json` carries the feasibility flag, the score, the violations, and any supporting evidence. A small but consequential detail is that some families mirror their case data in two places — inside `public_inputs` and at the top level of the task — and the artifact validator enforces that these copies agree, a check we added after a divergence between them slipped through once.

4.2 Agent-agnostic launch and harness robustness

Turning a task into something an arbitrary command-line agent can attempt, and then trusting the result enough to compare agents against one another, is more delicate than it first appears, and three properties of the harness are what make the comparison defensible. `prepare_agent_tasks.py` packages each task into a self-contained directory — the task and a short instruction file, with sample answers and any oracle-bearing fields stripped — and `run_agent_command.py` runs one shell command per packet, passing the relevant paths through environment variables under a per-case timeout.

The first property is isolation. Each case runs in a sandbox copy of its packet placed outside the repository, with that sandbox as the working directory and with no environment variable or import path pointing back at the repository root. This was not a precaution taken in the abstract: in an earlier design the packets lived inside the repository tree and an agent reached the evaluator and graded itself against the leaked oracle (Section 8.3), and relocating the sandbox is what closes that path. The second property is robustness to malformed submissions. A structurally valid but meaningless answer — nulls, strings where numbers belong, the wrong shape entirely — returns `feasible=false` with per-field violations instead of crashing the run, a behavior we verified by probing every one of the 41 families and that a field report had earlier shown to matter when an agent returning nulls

aborted a run at case 27 of 41. The third property is the separation of failure classes. A narrow retry policy distinguishes transient infrastructure failures — provider and transport errors, early silent aborts, quota and spend-limit signatures — from the failures that actually measure an agent, and retries only the former; quality failures, broken contracts, and per-case budget timeouts are never retried, because retrying them would quietly convert the benchmark into best-of-N. Each result records its attempt count, its initial failure class, the selected attempt, and the raw first attempt, and the run summary reports raw and retry-normalized metrics together with a count of unresolved infrastructure failures, so that a flaky provider can never be mistaken for a capable agent.

4.3 Evaluator maturity ladder

The evaluators are deliberately allowed to mature in place, along a ladder that runs from a formula or graph surrogate, through a deterministic engineering model, to a simulator-backed hidden evaluator, and ultimately to a stochastic or large-scale scenario suite. The public evaluators described here occupy the first two rungs. What makes the ladder usable is precisely the contract of Section 4.1: because the task and submission formats are fixed independently of how the check is computed, an internal step — say, replacing a DC power-flow surrogate with a pandapower AC solve — changes an evaluator without disturbing anything an agent sees.

4.4 The hidden split as a generator, not as files

The most distinctive architectural choice concerns the hidden split, which we do not store at all. The `tasks/*/hidden/` directory is excluded from version control; in its place the repository ships `generate_hidden_cases.py`, a registry of deterministic, physics-aware perturbations, one per family. A hidden case is produced from the public `case_001` by redrawing its numerical content — network parameters, weather series, load profiles, a fault location together with the indicator readings and complaints it implies — under a private seed, and the draw is accepted only if the family baseline still solves it and the evaluator confirms the solution feasible, with a deterministic redraw otherwise. The operator of a leaderboard keeps the seed private, but anyone holding the repository can read exactly how hidden cases are constructed without learning the particular numbers of any round.

5. Task Set

The public benchmark consists of 41 task families, which we group into eight domain areas to make the coverage legible: network analysis and operating constraints (`ybus_construction`, `pf_voltage_repair`, `contingency_screening`, `corrective_transmission_switching`, `dynamic_thermal_rating`); short circuit and protection (`three_phase_short_circuit`, with the IEC 60909 max/min current pair, `earth_fault_calculation`, `breaker_relay_short_circuit`, `distance_protection_settings`, `overcurrent_protection`, `cable_ampacity_sizing`); stability, grid code, and inverter-based resources (`critical_clearing_time` by the equal-area criterion, `transient_stability_prediction`, `frt_compliance`, `ibr_short_circuit_frt`, `min_synchronous_share`); DER, PV, EV, and storage (`pv_volt_var`, `ev_v2g_outage_schedule`, `ev_v2g_voltage_support`, `bess_ancillary_response`, `commercial_pv_lcoe_uncertainty`); microgrids and dispatch (`microgrid_economic_dispatch`, `rolling_microgrid_dispa`, `islanded_microgrid_pq_dispatch`, `dispatch_uncertainty`, `hydro_thermal_storage_uc`, `n1_generation_commitment`); reliability and restoration (`flisr_restoration`, `fault_section_localization`, `fci_placement`, `fci_saidi_caidi`, `operator_breaker_load_actions`); power quality, standards, assets, and cybersecurity (`en50160_voltage_compliance`, `power_quality_event_classification`, `harmonic_ieee519_compliance`, `transformer_thermal_loading` by IEC 60076-7, `fdi_state_estimation` by WLS with the largest-normalized-residual test, `protected_meter_placement`, `mas_frequency_handover`); and forecasting under uncertainty (`wind_power_forecast`, `wind_prediction_interval`). Every family carries a source card and a provenance entry with a confidence level, and the full catalog appears in the appendix.

5.1 Domain vetting of the set

Assembling tasks from the literature is not the same as making them correct, and a domain review removed four classes of factual defect, each recorded in the repository history. The clearest were physical: a RoCoF formula had placed system frequency in the denominator rather than the numerator — an `$f_0^2$` error — so that the inertia constraint it was meant to enforce never bound, and the fix restores the swing equation and rebalances the case. Others were conceptual: a task labeled "FDI detection" had shipped the attack labels among its inputs and collapsed to a threshold comparison, and we reworked it into a genuine weighted-least-squares state estimation with the iterative largest-normalized-residual test — a setting in which residual smearing makes the naive weighted test wrong — while scoping stealthy FDI explicitly out. A third kind was degenerate: critical clearing time was being read off precomputed stability flags in a table, and we replaced it with an analytic equal-area calculation. The last were overclaimed standards: an "IEC 60909" short-circuit calculation that omitted the voltage factor c and folded arc resistance into the maximum current understated that current by about ten percent in the non-conservative direction, and we reworked it into the standard's own max/min pair. Finally, three families that on inspection fell outside power engineering — two on building heating and one on Sobol variance decomposition — were retired in favor of domains the set had not yet covered: transformer thermal loading by IEC 60076-7, cable sizing by its three governing checks, and IEEE 519 harmonic compliance.

6. Evaluators and Metrics

What an evaluator returns is uniform across the benchmark — a feasibility flag, a score in the unit interval, a list of violations stated in words, and task-specific evidence — but how it arrives at a score depends on the kind of task. For an exact computation the score is binary, or graded by numeric tolerance. For a control or optimization task the score combines feasibility on the constraints with quality on the objective, and where the optimum is not unique — as in microgrid dispatch — the evaluator re-simulates the submitted decision and scores the gap to the optimal cost, so that an alternative optimum of equal cost earns full credit and a feasible but more expensive solution earns partial credit; this matters, because an early version that demanded an exact match to one hardcoded optimum penalized correct answers, a defect the agent matrix later exposed. Classification and localization tasks are scored against exact sets with enumerated misses, and standards-compliance tasks as pass/fail with diagnostic fields. At the level of a whole run we report two aggregates that need not agree — a feasible count, the share of tasks meeting the hard constraints, and a perfect count, the share scoring exactly one — and because Section 8 shows these can rank agents differently, we recommend reporting both alongside the mean score.

7. Reproducibility

The benchmark is meant to be re-run rather than taken on faith. A single entry point, `scripts/check.sh`, runs the unit-test suite over evaluators, baselines, the harness, and the generator; validates every artifact; evaluates all 41 public sample submissions one by one; and runs the public baselines. Those baselines are built on the same computational kernels as the evaluators and return 41 of 41 feasible at a mean score of 0.997 — a figure we are careful to present as a self-consistency check on the contract rather than as evidence that the tasks are easy. The distinction is not pedantic: as Section 8.2 shows, this very self-consistency is blind to errors in the evaluator, because a baseline derived from a faulty evaluator agrees with it by construction. Continuous integration runs the full check on every commit.

8. Experiments

We evaluated three command-line agents under identical prompts, a 600-second per-case timeout, and the transient-retry policy of Section 4.2, with no adaptation of the agents to the benchmark: Codex CLI 0.139.0 on gpt-5.5 at medium reasoning effort, the Cursor agent CLI on composer-2.5, and OpenCode 1.15.13 on deepseek-v4-flash.

8.1 Canonical round (public vs. hidden)

The reference round was run after every fix described in this paper. Each agent attempted the committed public `case_001` of every family and one freshly generated hidden `case_401` per family drawn from a new private seed; no run hit an unresolved infrastructure failure, and raw and retry-normalized metrics therefore coincide.

agent (model)	public feasible	public perfect	public mean	hidden feasible	hidden perfect	hidden mean
codex (gpt-5.5 medium)	40/41	36/41	0.971	40/41	37/41	0.969
cursor (composer-2.5)	39/41	35/41	0.947	40/41	36/41	0.969
opencode (deepseek-v4-flash)	37/41	35/41	0.900	38/41	35/41	0.920

Two things stand out. The benchmark discriminates: codex and cursor, both near a mean of 0.97, sit clearly above opencode at 0.90 to 0.92, and they do so on both splits. And, on clean specifications, the public and hidden splits are comparable — every agent's hidden score lands within noise of its public one. We report this plainly because an earlier, pre-fix development round had seemed to show the opposite, with the public leader collapsing on hidden cases; that divergence turned out to be an artifact of the specification defects of Section 8.2, which happened to fall hardest on one agent, and it disappeared once those defects and the evaluator bug were corrected. We therefore make no claim of a one-round difficulty gap between public and hidden. The case for a hidden split is instead prospective: a published set ages as models train on it, and the generator is what keeps a fresh, uncontaminated draw available. As mild corroboration, these recent models do not score higher on the published public set than on freshly generated hidden cases — the signature one would expect from memorization is simply absent at this case size.

8.2 Failure taxonomy: capability versus specification

Reading the failures across agents, rather than per agent, turned out to be the most useful thing we did, because the failures separate cleanly into two kinds. When agents fail a task in different ways — opencode breaching a voltage limit in Volt-VAR control where the others hold it, a wrong fault-ride-through verdict, an inverted sign on an EV discharge, divergent errors in a stability risk score — the disagreement is signal, and it is what a leaderboard should reward. But when every agent fails a task in the same way, the more likely explanation is that the task, not the agents, is wrong. Treating that pattern as a diagnostic surfaced ten specification defects over two passes: a field whose description contradicted its name, so that two agents correctly computed a fraction where the evaluator wanted a percentage; nested keys that did not match the checker; a semantically empty zero entry that was nonetheless rejected; the demand for an exact match to one of several equal-cost optima already mentioned; unstated conventions for a wind power curve, for quantiles, and for the terms of a conductor heat balance; a hardcoded dispatch policy; and an unguessable risk-score formula. In each case the fix was confirmed by rerunning the affected family, or by re-scoring the existing submissions, and watching the agents recover. We propose the underlying rule — an identical failure of all participants should be suspected as a defect in the specification or the evaluator — as a routine quality-control step for executable benchmarks.

The most consequential instance pointed not at a task but at an evaluator, and it is worth telling in full because it is the sharpest evidence we have for the value of the method. On the hidden draw of

`fault_section_localization`, all three agents placed the fault on section s24, a conclusion consistent both with the fault-current indicators and with the pattern of outage complaints; the evaluator insisted on a different section that in fact contradicted the case's own indicator readings. The agents were right and the evaluator was wrong. The cause was an off-by-one in the evaluator's indicator model: it had placed the faulted section's downstream node on the fault path, so that an indicator at that node was predicted to see fault current it would not actually see, whereas an indicator registers fault current only when the fault lies strictly downstream of it. What makes this more than a fixed bug is that the benchmark's own safeguards could not have caught it. The baseline that the benchmark uses to self-check is derived from the same function as the evaluator, so baseline and evaluator agreed with each other and the public sample passed; the error was invisible from the inside and became visible only because three independent agents brought the correct physics to bear. An executable benchmark, in other words, can harbor a latent domain error in the very code that defines correctness, and an agent matrix is a way to find it — a caution we would extend to any benchmark whose reference solutions are derived from its evaluator.

8.3 Evaluation integrity: an agent that reached the oracle

A separate lesson came from auditing the run logs rather than the scores. On the transient-stability task, one agent — Cursor — had located and executed the evaluator and graded itself against the oracle, a fact its own log records in the words "evaluates as feasible with a score of 1.0." This was possible only because, in that earlier design, the packets lived inside the repository working tree and the harness launched agents with the repository as their working directory, so that the rule against consulting the evaluator existed only as a sentence in the instructions. Instruction-level isolation, the episode makes clear, is not isolation at all for an agent with autonomous filesystem access, and it is what drove the sandbox of Section 4.2. We report it not as an embarrassment but as a methodological result in its own right.

9. Limitations

Several limitations bound what these results support. The public cases are compact — networks of three to five buses, short time series — so they test whether an agent commands a method, not whether it scales to industrial data volumes. The evaluators are deterministic surrogates, and no claim of industrial AC accuracy is warranted until the simulator-backed checkers of the roadmap are in place. With one case per family per split, a ranking over 41 points is statistically fragile, though the generator makes enlarging the sample inexpensive. The baseline's 41-of-41 is a self-consistency check on the contract, not a lower bound on difficulty, and — as Section 8.2 made vivid — it cannot validate the evaluator. The seed of any hidden round is spent once the round is disclosed, so a citable leaderboard needs a fresh seed on an isolated host. The sandbox isolates by relocation rather than containerization, which removes leakage through relative paths and environment but not a determined search for the oracle by absolute path, so competitive use would want host-level containers. We run one configuration per agent and so do not estimate the variance of LLM sampling. Finally, an attempt to evaluate the maintainer's own model family on a shared-budget account proved infrastructurally infeasible — it simply exhausted the session budget — and was excluded; a clean self-evaluation would require a dedicated API key and an explicit conflict-of-interest statement.

10. Conclusion

We have brought executable evaluation, by now standard for software and tool-use agents, to electric power engineering, where evaluation had remained textual. The benchmark turns power-system tasks into checks that recompute physics, and on a matrix of current agents it discriminates between them consistently across its public and hidden splits. Its most transferable contribution is not any single task but a method: reading an identical failure of all agents as a defect signal, which over the course of this work found ten specification defects and a latent physics bug inside one of our own evaluators that the benchmark could not have caught by self-consistency

alone — a reminder that a baseline derived from an evaluator cannot certify it. The lasting value of the artifact lies in the contract, the domain-grounded task set, the checkers, the seed-based hidden generator with its prospective resistance to contamination, the isolated harness, and the reproducible quality-control loop; the road ahead is to fill the checkers with simulator-backed implementations without disturbing the contract that holds the whole thing together.

Appendix A. Task Catalog

See `docs/bilingual_task_inventory.md` (41 families, EN/RU, with `task_id` and paths).

Appendix B. Experiment Artifacts

- Canonical round: `runs/canonical_round1/` (per-agent, per-split summaries and logs).
- Earlier development matrices and the spec-fix history: `runs/public_agent_smoke/REPORT.md`, `runs/hidden_agent_smoke/REPORT.md`, and the repository commit history.